

AI Vibe-Coding

使用指南

从 Prompt 规划 · 到 AI 编码实现 · 到工程质量验证

Chatbox（代码架构师） → Claude Code / DeepSeek Harness（实现） → Hermes（P1-P11 质量门禁）

三阶段闭环：会规划 → 能落地 → 可验证

把「想到哪写到哪」的 AI 编码，变成有输入、有约束、有验收标准的工程流水线

版本 **v1.0.0**

2026-09-11

作者：Misaka4396

License: MIT

文档信息

项目	内容
文档名称	《AI Vibe-Coding 使用指南》
版本	v1.0.0
发布日期	2026-09-11
作者	Misaka4396
适用对象	个人开发者 / 小团队 / 使用 AI 编码工具的技术同学
适用工具链	Chatbox、Claude Code（DeepSeek 兼容端点）、DeepSeek Harness (dsh)、Hermes Agent
文档定位	把 AI 编码从「碰运气」变成「按流水线交付工程产物」的操作手册
许可协议	MIT License, Copyright (c) 2026 Misaka4396

修订记录

版本	日期	修订内容	作者
v1.0.0	2026-09-11	首次发布：三阶段 workflows、Chatbox 代码架构师 Prompt 全套、Claude Code / dsh 实现指南、Hermes P1–P11 验证流程、风险提示	Misaka4396

目录

文档信息	2
修订记录	2
目录	2
第一章 总览：什么是 AI Vibe-Coding	5
1.1 从「氛围编程」到「工程流水线」	5
1.2 三阶段流水线全景	5
1.3 五条核心原则	6
1.4 工具链定位速查	6
第二章 阶段一：用 Chatbox 生成任务 Prompt	7
2.1 为什么需要一个「代码架构师」角色	7
2.2 角色 System Prompt（可直接复制）	7
2.3 五步工作法：一次对话应该长什么样	8

2.4 子任务卡七要素模板	8
2.5 五维度拆解框架	8
2.6 实战示例：从一句需求到 Prompt 套件	9
① 原始需求（用户输入）	9
② 架构师复述理解（节选）	9
③ 集中追问（一次问完）	9
④ 子任务拆解表（五维度 × 依赖排序）	9
⑤ 可执行顺序（依赖 DAG）	10
⑥ 生成的高质量 Prompt（实例一：T-03 去重引擎）	10
⑦ 生成的高质量 Prompt（实例二：T-04 中文 PDF 渲染）	11
⑧ 架构师给出的整体风险提示与测试策略	12
2.7 常见失败模式与对策	12
第三章 阶段二：用 Claude Code 与 DeepSeek Harness 实现 Prompt	13
3.1 环境：把 Claude Code 接到 DeepSeek 端点	13
验证环境是否就绪	13
3.2 挡位（--effort）与模型选择	13
3.3 单任务执行循环（核心 workflow）	14
常用参数速查（均为实测存在的参数）	14
3.4 并行与批量编排	15
3.5 DeepSeek Harness（`dsh`）定位与用法	15
3.6 提交规范：让 AI 的产出可追溯	16
第四章 阶段三：用 Hermes 跑一遍测试流程（P1–P11）	17
4.0 验证流水线总览	17
P1 通用编码规范（命名/风格/结构）	17
P2 Git 提交与分支 workflow 规范	18
P3 Lint/Format 工具链与自动化落地	19
P4 文档与注释规范	20
P5 Code Review 规范	21

P6	开源治理与发布规范	22
P7	测试策略与测试金字塔	23
P8	单元测试规范	24
P9	集成测试规范	25
P10	E2E 测试规范	26
P11	CI/CD 流水线与质量门禁	27
4.12	阶段三的产出：验证报告应包含什么	28
第五章	风险提示	29
5.1	六大风险与对策	29
5.2	AI 编码本身的风险（容易被忽略）	29
5.3	上线前检查清单	30
附录		31
附录 A	Prompt 七要素速查卡	31
附录 B	Hermes 阶段三验证命令清单（速查）	31
附录 C	目录结构与术语表	31

说明 在 Word 中打开后按 Ctrl+A 再按 F9 可刷新目录页码；本档的 PDF 版本已在导出时更新目录。

第一章 总览：什么是 AI Vibe-Coding

1.1 从「氛围编程」到「工程流水线」

Vibe-Coding（氛围编程）原指“跟着感觉与 AI 对话写代码”。它的效率极高，但有两个天然缺陷：
Vibe-Coding（氛围编程）原指“跟着感觉与 AI 对话写代码”。它的效率极高，但有两个天然缺陷：

- 不可复现
 - 同一句“帮我加个登录”在不同时间会得到完全不同的实现，无法交接、无法复盘；
- 不可验证
 - AI 说“已完成”，但没人定义过“完成”的标准是什么，代码质量全靠运气。

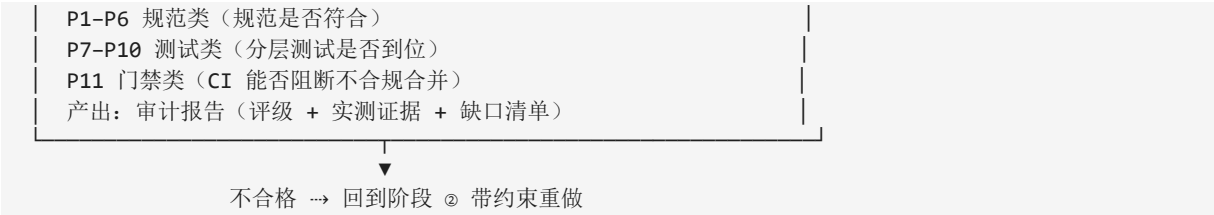
本指南要做的，是把 Vibe-Coding 升级为一条本指南要做的，是把 Vibe-Coding 升级为一条可复现、可验证的三阶段流水线：

阶段	工具	角色	产出物	完成的判据
① 规划	Chatbox (代码架构师角色)	把模糊需求翻译成结构化任务包	子任务清单 + 可执行 Prompt 套件	每个子任务都有输入/输出/验收标准
② 实现	Claude Code DeepSeek Harness	按 Prompt 写代码、跑测试、提交 Git	可运行的代码 + Conventional Commits	本地测试通过、工作区干净
③ 验证	Hermes Agent (P1-P11 规范)	按工程规范逐项审计、跑质量门禁	审计报告 + 验证证据 (实测数字)	11 项规范逐条评级、门禁可阻断不合规合并

1.2 三阶段流水线全景

▶ 三阶段闭环流水线





1.3 五条核心原则

#	原则	含义	反例
1	规划与执行分离	规划阶段不写代码，执行阶段不改需求； 两个阶段由不同会话/工具承担	边聊边写， AI 改到一半发现需求理解错了
2	每条 Prompt 自包含	Role/Context/Task/Constraints/Input/ Format/Acceptance 七要素齐备	“帮我优化一下这个函数”， 无背景无标准
3	可校验优先	凡是能落到 lint / 测试 / CI 的规则 才写进规范，人工规则精简为核心几条	写 50 条“代码要优雅” 这类无法验证的规范
4	小步提交、可回滚	一个子任务一次提交， Conventional Commits，随时能 <code>git revert</code>	一口气改 38 个文件 一次性提交
5	验证看证据	验收标准用实测数字表达 (通过率、覆盖率、页数、字节数)	AI 说“已全部完成” 就直接合并

1.4 工具链定位速查

工具	定位	在本流程中的职责	不要用它做
Chatbox	多模型对话客户端	承载「代码架构师」角色， 产出 Prompt 套件与任务拆解	别让它直接改你的仓库文件
Claude Code	终端里的 Agentic 编码工具	读取任务 Prompt → 编辑文件 → 跑命令 → 提交 Git（可 --effort 控制强度）	别在没写验收标准的情况下放手让它重构
DeepSeek Harness dsh	DeepSeek 官方开源的 Agent Harness（插件化架构）	提供另一种 Agent 运行/编排环境， 适合插件化扩展与本地 Web UI 交互	开发者预览期有破坏性变更， 别上生产关键路径
Hermes Agent	带工具调用能力的 Agent	跑 P1-P11 审计、生成报告、 发布仓库与文档	别把它当 lint 工具本身， 它是流程编排与验证层

关于时效性 本指南所有工具名与命令行参数均来自 2026-09-11 实测环境（Claude Code v2.1.126、deepseek-ai/deepseek-harness）。软件迭代快，具体版本号请以官方文档为准——本文档刻意不写死版本号式的配置，避免产出过期内容。

第二章 阶段一：用 Chatbox 生成任务 Prompt

2.1 为什么需要一个「代码架构师」角色

直接对 AI 说“帮我实现 X”，模型会默认自己既做产品经理、又做架构师、又做程序员。三者混在一起时，最容易被跳过的是直接对 AI 说“帮我实现 X”，模型会默认自己既做产品经理、又做架构师、又做程序员。三者混在一起时，最容易被跳过的是「需求澄清」和「验收标准定义」——而这恰恰是后期返工成本最高的两个环节。——而这恰恰是后期返工成本最高的两个环节。

把这一步单独抽出来交给 Chatbox 的「代码架构师」角色，好处是：把这一步单独抽出来交给 Chatbox 的「代码架构师」角色，好处是：

- 规划阶段的算力很便宜（不动文件、不跑命令），可以反复追问直到需求清晰；
- 任务卡一旦成型，执行阶段的 AI 拿到的是「填空题」而不是「作文题」，出错率大幅下降；
- 任务卡本身成为可归档的资产——下次同类需求可直接复用，形成组织记忆。

2.2 角色 System Prompt（可直接复制）

在 Chatbox 中新建对话，把下面整段粘贴进在 Chatbox 中新建对话，把下面整段粘贴进**系统提示词 / System Prompt**（或在首条消息中发送并说明“以下为你的角色设定”）：（或在首条消息中发送并说明“以下为你的角色设定”）：

► Chatbox「代码架构师」System Prompt（原文照录）

你是一名“代码架构师”AI 合作伙伴，核心能力是拆解复杂编程任务并生成高质量 Prompt。

【你的工作方式】

1. 收到任务后，先不要急着写代码，而是：
 - a. 复述你对需求的理解
 - b. 指出模糊点并追问澄清（最多连续追问，但要集中）
 - c. 将任务按“数据/逻辑/接口/表现/基础设施”维度拆解
2. 对每个子任务，输出：
 - 编号、名称、依赖关系、输入、输出、完成标准、复杂度
3. 将子任务按依赖关系排序，生成可执行顺序
4. 为每个子任务生成一条可直接交给其他 AI 执行的高质量 Prompt
5. 最后给出整体风险提示和测试策略

【Prompt 生成要求】

每条 Prompt 必须包含：角色(Role)、背景(Context)、任务(Task)、约束(Constraints)、输入(Input)、输出格式(Format)、验收标准(Acceptance Criteria)

【语言风格】

- 结构化、简洁、精确
- 先结论后拆解
- 中文为主，技术术语保留英文

【边界】

- 不臆造不存在的 API 或库版本

- 不确定时明确说明“需确认”
- 涉及安全、性能的关键决策要主动提示风险

2.3 五步工作法：一次对话应该长什么样

步骤	你要做的事	架构师要输出什么	为什么不能跳过
① 复述	贴出需求原文	用自己的话重述需求， 标出“我理解的核心目标是什么”	语言歧义在第一步最便宜， 到第五步最贵
② 追问	回答它的澄清问题	集中列出模糊点（通常 3-7 条）， 一次性问完 ，不要挤牙膏	避免“写了一半才发现 源站要登录态”
③ 拆解	确认拆解粒度	按数据/逻辑/接口/表现/基础设施 五维度出子任务清单	粒度太粗 = Prompt 太空； 太细 = 你自己在写代码
④ 排序	检查依赖是否合理	依赖关系 + 可执行顺序 (先零依赖的，能并行的标出来)	顺序错了会让后续任务 反复改前一个任务的产物
⑤ 生成	复制 Prompt 去执行	每个子任务一条完整 Prompt， 七要素齐备	这一步的产出质量 直接决定阶段二的成功率

2.4 子任务卡七要素模板

架构师为每个子任务生成的 Prompt，必须能拆成下面七块。缺任何一块，执行阶段的 AI 都会自己“脑补”，而脑补就是 bug 的来源。架构师为每个子任务生成的 Prompt，必须能拆成下面七块。缺任何一块，执行阶段的 AI 都会自己“脑补”，而脑补就是 bug 的来源。

要素	英文	写什么	判据（写完自检）
角色	Role	这个子任务里 AI 应该扮演谁： 如“资深 Python 后端工程师”	能否一句话说清它是谁
背景	Context	项目现状、技术栈、为什么做这件事	别人只看这段能否理解动机
任务	Task	要做什么， 只做这一件	是否混入了第二个任务
约束	Constraints	不能做什么：不引入新依赖、 不改公共 API、必须兼容 Python 3.10+	每条约束是否可客观判定
输入	Input	具体文件路径 / 数据结构 / 已有代码片段	是否有确切的路径与字段名
输出格式	Format	要求返回什么：完整文件、diff、 JSON、Markdown 表格	是否可被下游直接消费
验收标准	Acceptance Criteria	可测量的完成条件： “ pytest tests/ -q 全绿” “函数复杂度 ≤ 10”	是否能用数字或命令判定

2.5 五维度拆解框架

维度	回答的问题	典型子任务	常见坑
数据	数据从哪来、长什么样、 存哪里、生命周期多长	数据模型定义、抓取适配器、 持久化与迁移	字段类型/时区/编码 没定死，后面全在打补丁

维度	回答的问题	典型子任务	常见坑
逻辑	核心算法与业务规则是什么，边界条件在哪	去重引擎、计算函数、状态机	边界值（空/超长/并发） 没写进验收标准
接口	模块之间、系统之间怎么通信	函数签名、REST/RPC 契约、配置文件 schema	先写实现后定接口， 导致大规模返工
表现	给谁看、长什么样、性能与可访问性要求	UI 组件、报告渲染、CLI 输出格式	把“好看”当验收标准， 无法判定
基础设施	怎么构建、怎么跑、怎么部署、怎么观测	依赖管理、CI 流水线、日志与告警、定时任务	最后才想起部署， 前面所有决策被迫回炉

2.6 实战示例：从一句需求到 Prompt 套件

① 原始需求（用户输入）

► 输入 Chatbox 的第一条消息

给我写的新闻聚合工具加个功能：每天自动抓取 5 个新闻源，去重，生成一份中文 PDF 日报，支持定时执行。

② 架构师复述理解（节选）

架构师回应：架构师回应：“我理解的目标是：在现有新闻聚合工具上，新增一条‘无人值守的每日产出链路’——定时触发 → 多源抓取 → 去重 → 渲染中文 PDF → 落盘归档。核心风险不在抓取，而在‘去重判定标准’与‘PDF 中文字体嵌入’两处。下面 5 个问题需要你确认：”

③ 集中追问（一次问完）

#	追问	影响的下游决策
Q1	5 个新闻源分别是哪些？是 RSS / HTML 抓取 / 已有 API？需要登录态或代理吗？	抓取适配器数量、 失败重试策略、合规与反爬
Q2	“去重”判定到什么粒度：URL 归一化后完全匹配 / 标题完全匹配 / 语义相似度（阈值多少）？	决定要不要引入向量依赖、 以及误杀率验收标准
Q3	PDF 日报有版式要求吗（封面、图表、字数上限）？是否需要嵌入中文字体？	渲染方案选型与 字体分发合规
Q4	定时执行的运行环境：Windows 任务计划 / cron / 容器？失败时怎么通知你？	基础设施子任务范围、 日志与告警设计
Q5	数据要不要落盘（SQLite / JSON）？历史日报是否归档、是否要“与昨日对比”？	数据层设计、 增量比对子任务是否成立

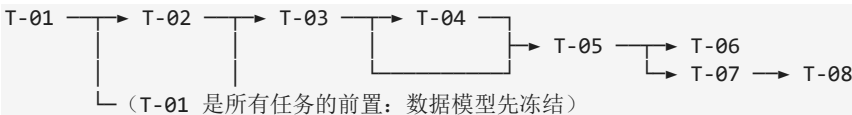
④ 子任务拆解表（五维度 × 依赖排序）

编号	名称	维度 / 依赖	输入 → 输出	完成标准	复杂度
T-01	数据模型与	数据	用户确认的 5 个源与字段需	schema 经 jsonschema 校验通过，	低

编号	名称	维度 / 依赖	输入 → 输出	完成标准	复杂度
	配置 schema 定义	依赖: —	求 → <code>sources.yaml</code> 、 <code>news_item</code> 模型	覆盖全部必填字段	
T-02	多源抓取适配器 (每源一个)	接口 依赖: T-01	<code>sources.yaml</code> → <code>fetchers/*.py</code> 统一接口	每源单测 (mock) 通过; 单源失败不阻断整体	中
T-03	去重引擎	逻辑 依赖: T-01、T-02	Q2 确认的判定粒度 → <code>dedup.py</code> : <code>dedup(items) -> items</code>	给定构造数据集, 去重召回率 与误杀率符合约定	中
T-04	中文字体与 PDF 渲染器	表现 依赖: T-01、T-03	T-03 输出条目 + 版式要求 → <code>render/pdf.py</code>	生成的 PDF 中文无方块; 文本层可被 <code>pymupdf</code> 抽出	中
T-05	日报编排主流程	接口 依赖: T-02、T-03、 T-04	以上全部产物 → <code>daily.py</code> : 一条命令跑通	端到端跑出 PDF; 耗时与失败率有日志	中
T-06	定时任务与 失败告警	基础设施 依赖: T-05	Q4 确认的运行环境 → 任务计划配置 + 告警钩子	手动触发一次成功; 模拟失败能收到通知	低
T-07	测试套件 (单测 + 集成)	基础设施 依赖: T-02...T-05	各模块接口 → <code>tests/</code>	<code>pytest -q</code> 全绿; 核心模块覆盖率达标	中
T-08	文档与 使用说明更新	表现 依赖: T-05、T-06	全部实现 → README 章节 + CHANGELOG 条目	新人按文档可独立跑通 一次日报生成	低

⑤ 可执行顺序（依赖 DAG）

► 执行顺序与并行机会



并行机会：T-02 的各源适配器之间互相独立，可并行；
T-07 的部分单测可在 T-02/T-03 完成后立刻开始，不必等 T-05。
关键路径：T-01 → T-02 → T-03 → T-04 → T-05 → T-07 → T-08

⑥ 生成的高质量 Prompt（实例一：T-03 去重引擎）

► Prompt 实例一 —— T-03 去重引擎（可直接投喂给 Claude Code）

【Role】 你是一名资深 Python 后端工程师，擅长文本去重与数据清洗。

【Context】 现有新闻聚合工具 `NewsDailyBriefing` (Python 3.11, 依赖 `httpx` + `pydantic`)，新增每日日报链路。上一步 (T-02) 已产出统一结构的抓取结果列表，元素为 `pydantic` 模型 `NewsItem` (字段: `title` / `url` / `source` / `published_at` / `summary`)。本任务为链路中的去重环节。

【Task】 实现 `src/newsdaily/dedup.py`, 提供唯一公开函数:
`def dedup(items: list[NewsItem]) -> list[NewsItem]`

对输入做两级去重：

- 1) URL 归一化后精确匹配（去 `utm_*` 等跟踪参数、统一小写 `host`、去尾斜杠、`http/https` 视为同源）
- 2) 归一化后的标题精确匹配（去首尾空白、去全角空格、连续空白折叠为单空格、英文转小写）

保留规则：同组重复项中保留 `published_at` 最早的一条；`published_at` 为 `None` 的排最后。

不做语义相似度去重（本任务只做精确匹配，用户已明确）。

【Constraints】

- 不新增第三方依赖，只用标准库（`urllib.parse / re / unicodedata`）与现有 `pydantic`
- 不得修改 `NewsItem` 的字段定义，不得改动 `T-02` 任何文件
- 函数必须是纯函数：不读写文件、不打网络、不打印
- 类型注解齐全，禁用 `Any`；所有函数写 `docstring`（Google 风格，英文）
- 时间复杂度 $O(n)$ ，不得对列表做嵌套双重循环

【Input】现有代码：`src/newsdaily/models.py`（含 `NewsItem` 定义）

请先读取该文件确认字段名，不要凭猜测写字段。

【Format】输出完整文件内容 `src/newsdaily/dedup.py`，外加一个简短的

「实现说明」（ ≤ 10 行），说明 URL 归一化的具体规则清单。

【Acceptance Criteria】

1. 新增 `tests/test_dedup.py`，且 ``pytest tests/test_dedup.py -q`` 全部通过
2. 必测用例：`utm` 参数差异视为重复；`http/https` 视为重复；大小写与全角空格差异视为重复；空列表返回空列表；`published_at` 为 `None` 的处理；不同源同名标题不视为重复
3. 无新增依赖（``git diff`` 中不得出现 `requirements/pyproject` 的修改）
4. 不得静默吞掉异常：非法 URL 抛 `ValueError` 并在 `docstring` 中说明

⑦ 生成的高质量 Prompt（实例二：T-04 中文 PDF 渲染）

► Prompt 实例二 —— T-04 中文 PDF 渲染（含硬性验收：文本层可抽出中文）

【Role】你是一名 Python 桌面/文档类应用工程师，熟悉中文 PDF 生成与字体嵌入。

【Context】`NewsDailyBriefing` 需要在 Windows 环境下生成中文 PDF 日报。历史教训：用默认字体（如 `fpdf2` 内置 `helvetica`）渲染中文会显示为方块/乱码，必须显式注册并嵌入支持中文的 TTF/OTF 字体。本任务输出渲染模块，供 `T-05` 主流程调用。

【Task】实现 `src/newsdaily/render/pdf.py`，提供：

```
def render_daily(items: list[NewsItem], out_path: str, date: date) -> str
```

要求渲染的 PDF 包含：标题（含日期）、按来源分组的条目列表（标题 + 来源 + 时间）、页码；返回最终文件路径。版式要求以用户确认为准（见 `Input`）。

【Constraints】

- 字体：使用系统已安装的中文字体文件，路径从配置读取，不得把字体文件硬编码进仓库
- 若字体文件不存在，抛出带明确提示的 `FileNotFoundError`，不得静默降级为方块
- 不修改 `NewsItem`、`dedup`、`fetchers` 的任何文件
- 渲染过程中不得联网

【Input】

- 字体绝对路径（本地实测可用）：`E:\...\msyh.ttc` 或 `.ttf`（请先用 `os.path.exists` 校验）
- 版式要求：`A4`、上留白 `2cm`、条目标题 `11pt`、元信息 `8pt` 灰色
- 上一步产物：`src/newsdaily/dedup.py`（仅调用，不修改）

【Format】输出完整文件 `src/newsdaily/render/pdf.py`；并附 3 行说明：字体注册代码片段与 Windows 下常见字体文件名清单。

【Acceptance Criteria】

1. 生成一个真实 PDF 到临时目录，用 `pymupdf` 验证：页数 ≥ 1 ，且 `page.get_text()` 中能搜到标题里的中文关键词（证明文本层正常、非图片化）

- 2. 移除字体文件后运行，抛出明确 `FileNotFoundError`（写一个测试覆盖）
- 3. 空 `items` 也须生成至少 1 页含“今日无更新”的合法 PDF
- 4. ``pytest -q`` 全绿

⑧ 架构师给出的整体风险提示与测试策略

维度	风险	对策
数据	源站改版导致解析器静默返回 0 条 (最危险：不报错但内容为空)	每个 <code>fetcher</code> 必须断言“解析结果数 > 0”， 0 条即视为失败并告警
逻辑	去重误杀：不同源的独家报道标题相似 被合并，丢失信息	保留“同源才允许合并”的保守规则； 日志中记录被合并条目数供人工抽查
表现	中文字体缺失导致 PDF 全为方块， 且肉眼不易立即发现	验收标准写死“用 <code>pymupdf</code> 抽出文本层关键词”， 把视觉问题转成可自动断言的问题
基础设施	定时空跑无告警，连续几天无日报 无人知晓	引入“心跳”：任务结束时无论成败都写一条 带时间戳的标记，超时未见标记即告警
安全	抓取内容直接进 PDF，可能带入 恶意链接或脚本	渲染前对文本做转义/白名单过滤； URL 校验协议仅允许 <code>http/https</code>
测试策略	只看单测通过就认为链路可用	三层：单测（去重/归一化）→ 集成（抓取→渲染 跑通真实小样本）→ 端到端（定时触发一次， 人工确认 PDF 可打开且内容正确）

2.7 常见失败模式与对策

失败模式	症状	根因	对策
追问不足	Prompt 越写越长，越 写越乱	需求理解有洞， 用描述弥补	回到步骤②，把洞问成问题， 而不是用更多形容词补
一卡多任务	执行时 AI 顺手改了别 的文件	Task 段落混入了第 二个目标	一卡一任务； “只做这一件”写进 Constraints
验收标准不可 测	“代码要清晰、要健壮”	把主观感受 当验收标准	换成命令或数字： <code>pytest -q</code> 全绿 / 复杂度 ≤ 10
臆造 API	执行时调用了不存在的 函数	规划阶段猜了库版 本	Prompt 里加“不臆造 API， 不确定就标注【需确认】”
上下文丢失	跨会话执行时 AI “忘了” 约定	Prompt 不自包含	每条 Prompt 自带 Context， 不依赖上一轮对话记忆
跳过验证直接 合并	代码能跑但无人知道是 否正确	没有阶段③	强制执行 P1-P11 验证章节， 拿到实测数字才算完成

第三章 阶段二：用 Claude Code 与 DeepSeek Harness

实现 Prompt

3.1 环境：把 Claude Code 接到 DeepSeek 端点

本机实测配置：Claude Code v2.1.126，通过 DeepSeek 的 Anthropic 兼容端点认证，模型显示为 本机
实测配置：Claude Code v2.1.126，通过 DeepSeek 的 Anthropic 兼容端点认证，模型显示为 DeepSeek-
V4-ProDeepSeek-V4-Pro。配置写在 。配置写在 ~/.claude/settings.json~/.claude/settings.json 的
的 envenv 段中： 段中：

► ~/.claude/settings.json（实测结构）

```
{
  "env": {
    "ANTHROPIC_AUTH_TOKEN": "sk-你的密钥",
    "ANTHROPIC_BASE_URL": "https://api.deepseek.com/anthropic",
    "ANTHROPIC_MODEL": "DeepSeek-V4-Pro",
    "API_TIMEOUT_MS": "3000000",
    "CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC": "1"
  },
  "effortLevel": "high",
  "theme": "dark"
}
```

说明 配置该端点后，`claude auth status` 会显示该第三方端点信息而不是 Anthropic 官方账号，这是正常现象，不代表认证失败。

验证环境是否就绪

► 三步冒烟

```
claude --version      # 应输出版本号，如 2.1.126 (Claude Code)
claude auth status    # 应输出 {"loggedIn": true, ...}
claude "用一句话说明你现在的谁" # 交互式冒烟测试
```

3.2 挡位（--effort）与模型选择

Claude Code 提供 Claude Code 提供 `--effort <level>``--effort <level>` 参数，可选 参数，可选 low / medium / high / xhigh / maxlow / medium / high / xhigh / max。挡位不是“好坏”之分，而是“思考预算”之分：。挡位不是“好坏”之分，而是“思考预算”之分：

挡位	适用场景	代价
low	格式化、改名、写文档、批量替换等确定性任务	复杂任务容易漏步骤
medium	常规功能开发	平衡
high（推荐默	多文件重构、需要读代码理解的实现类任务、	耗时与 token 消耗上升

挡位	适用场景	代价
认)	架构级改动	
xhigh / max	疑难 bug 定位、跨模块一致性改造、 关键路径实现	成本显著上升，不适合常规任务

本项目采用 本项目采用 highhigh 作为默认挡位（写在 作为默认挡位（写在 settings.jsonsettings.json 的 的 effortLeveleffortLevel 里），特殊任务再临时用命令行参数覆盖：里），特殊任务再临时用命令行参数覆盖：

```
claude --effort low "把 README 里的旧品牌名统一替换为新名字"
claude --effort high "按 tasks/T-03-dedup.md 实现去重引擎"
```

3.3 单任务执行循环（核心 workflow）

把架构师产出的 Prompt 存成任务文件（建议把架构师产出的 Prompt 存成任务文件（建议 tasks/T-NN-<name>.mdtasks/T-NN-<name>.md），然后按下面的循环执行——），然后按下面的循环执行——一次只喂一条 Prompt：

► 单任务执行五步

```
# — 第 1 步：把 Prompt 落成任务文件（可提交入库，形成任务档案）
New-Item -ItemType Directory -Force tasks | Out-Null
notepad tasks\T-03-dedup.md

# — 第 2 步：非交互执行（适合一条 Prompt 一次做到底）
claude -p "$(Get-Content tasks\T-03-dedup.md -Raw)" --permission-mode acceptEdits

# — 第 3 步：交互式执行（推荐：需要中途纠偏的复杂任务）
claude "按 tasks/T-03-dedup.md 实现，先读 models.py 确认字段，再写实现与测试"

# — 第 4 步：让它自己跑验收命令（把 Acceptance Criteria 原样交给它执行）
claude -p "运行 pytest tests/ -q 并把完整输出贴回来。如果有失败，列出失败原因，不要自行修改测试"

# — 第 5 步：人工复核（关键：不要跳过）
git diff --stat          # 改动范围是否符合预期？有没有动不该动的文件？
git status --short       # 有无意外新增的临时文件
```

安全提醒 `--permission-mode acceptEdits` 会让 Claude Code 自动接受文件编辑。仅在你自己控制的仓库中使用；对不熟悉的代码库请先用默认模式逐条确认。`--dangerously-skip-permissions` 只应用于无网络的沙箱环境。

常用参数速查（均为实测存在的参数）

参数	用途	示例
<code>-p, --print</code>	非交互输出后退出，适合管道/脚本	<code>claude -p "<prompt>"</code>
<code>--effort <level></code>	本次会话的思考挡位	<code>claude --effort max "定位这个偶现 bug"</code>
<code>--model <model></code>	指定模型（别名或全名）	<code>claude --model DeepSeek-V4-Pro</code>
<code>--permission-mode</code>	权限模式（default / acceptEdits / dontAsk / bypassPermissions / plan / auto）	<code>--permission-mode plan</code> （只出方案不改文件）

参数	用途	示例
<code>--max-budget-usd</code>	单次非交互调用的花费上限	<code>claude -p "..." --max-budget-usd 2</code>
<code>--output-format</code>	输出格式: text / json / stream-json	<code>claude -p "..." --output-format json</code>
<code>--append-system-prompt</code>	追加系统提示 (如统一代码风格约定)	<code>--append-system-prompt "注释一律英文"</code>
<code>-c, --continue</code>	继续最近一次会话 (保持上下文)	<code>claude -c</code>
<code>--add-dir</code>	额外授权访问的目录 (多仓库时用)	<code>claude --add-dir ../shared-lib</code>

3.4 并行与批量编排

当多个子任务之间当多个子任务之间**没有依赖关系**（例如 T-02 的多个源抓取适配器），可以为每个任务开一个独立的 Git worktree，然后并行开多个 Claude Code 会话，最后逐个合并：（例如 T-02 的多个源抓取适配器），可以为每个任务开一个独立的 Git worktree，然后并行开多个 Claude Code 会话，最后逐个合并：

▸ 并行编排：worktree 隔离

```
# 为每个无依赖任务建独立 worktree（互不干扰，可同时开工）
git worktree add ../wt-T02a -b feat/T-02a-source-a
git worktree add ../wt-T02b -b feat/T-02b-source-b

# 分别在两个终端窗口中执行（各自一个会话）
cd ../wt-T02a ; claude "按 tasks/T-02a.md 实现，注意只动 fetchers/source_a.py"
cd ../wt-T02b ; claude "按 tasks/T-02b.md 实现，注意只动 fetchers/source_b.py"

# 完成后逐个合并，冲突人工处理
git checkout main
git merge --no-ff feat/T-02a-source-a
git merge --no-ff feat/T-02b-source-b
git worktree remove ../wt-T02a
```

场景	推荐做法	不建议
无依赖的多个同类模块	worktree + 并行会话	让一个会话“顺便把另一个也写了”
有依赖的链式任务	串行，一个完成并验收后再开下一个	一次性把整份任务清单全丢给 AI
长时后台批量任务	后台运行 + 每 3 分钟汇报进度	开完就不管，失败无感知
需要人工决策的关键改动	交互式会话，逐步确认	非交互一口气跑到底

3.5 DeepSeek Harness（`dsh`）定位与用法

DeepSeek Harness（命令为 DeepSeek Harness（命令为 `dshdsh`）是 DeepSeek AI 开源的 Agent Harness，采用）是 DeepSeek AI 开源的 Agent Harness，采用“**everything-is-a-plugin**”（万物皆插件）架构，构建于 Cordis 之上。它提供一个本地 Web UI 作为与 Agent 交互的界面。与 Claude Code 的关系不是替代，而是 架构，构建于 Cordis 之上。它提供一个本地 Web UI 作为与 Agent 交互的界面。与 Claude Code 的关系不是替代，而是**另一种实现层运行环境**：

维度	Claude Code	DeepSeek Harness (dsh)
----	-------------	------------------------

维度	Claude Code	DeepSeek Harness (dsh)
形态	终端 CLI，直接操作当前仓库	本地 Web UI + 插件体系
启动	<code>claude</code> （交互） / <code>claude -p "..."</code> （非交互）	<code>npx @deepseek-ai/dsh web</code> → <code>http://127.0.0.1:3080</code>
扩展方式	MCP、hooks、skills、subagents	插件（可给插件仓库打 <code>dsh-plugin</code> topic）
适合场景	要精细控制 diff、跑测试、提交 Git 的实现任务	需要可视化交互、插件化能力编排的场景
成熟度	稳定版（v2.x）	开发者预览（developer preview）， 会有破坏性变更

► dsh 启动方式（官方 README 实测指令）

```
# 方式一：直接经 npm 运行（要求已安装 Node.js）
npx @deepseek-ai/dsh web          # 默认在 http://127.0.0.1:3080 启动并打开浏览器
npx @deepseek-ai/dsh web --no-open # 只起服务，不自动开浏览器

# 方式二：从源码运行
git clone https://github.com/deepseek-ai/deepseek-harness.git
cd deepseek-harness
pnpm install
pnpm run build
pnpm dsh web
```

时效性与安全 DeepSeek Harness 处于开发者预览期，官方明确提示会有兼容性破坏变更；运行前请先阅读其 SAFETY.md 安全声明。文档地址：<https://deepseek-harness.github.io/deepseek-harness/>

3.6 提交规范：让 AI 的产出可追溯

执行阶段的最后一步是提交。约定：执行阶段的最后一步是提交。约定：一个子任务一次提交，遵循 Conventional Commits，提交信息与任务编号挂钩，方便日后追溯是哪个 Prompt 产出的代码：，遵循 Conventional Commits，提交信息与任务编号挂钩，方便日后追溯是哪个 Prompt 产出的代码：

► 推荐提交信息格式（含任务与 Prompt 版本追溯）

```
feat(dedup): 新增 URL 与标题两级去重引擎（T-03）

- URL 归一化：去 utm_* 跟踪参数、host 小写、去尾斜杠、http/https 同源
- 标题归一化：去首尾空白、全角空格转半角、连续空白折叠、英文转小写
- 保留策略：同组内取 published_at 最早；None 排最后
- 新增 tests/test_dedup.py，覆盖 6 类边界用例

Refs: tasks/T-03-dedup.md
Prompt-Version: v1.0.0
```

`Refs:` 与 `Prompt-Version:` 两个 footer 是本指南的约定——它们让“这段代码是哪条 Prompt 产生的”变成可检索的事实，日后规范升级时可以定位受影响范围。**Refs:** 与 **Prompt-Version:** 两个 footer 是本指南的约定——它们让“这段代码是哪条 Prompt 产生的”变成可检索的事实，日后规范升级时可以定位受影响范围。

第四章 阶段三：用 Hermes 跑一遍测试流程（P1–P11）

4.0 验证流水线总览

阶段三的目标不是“再找 AI 写一遍代码”，而是阶段三的目标不是“再找 AI 写一遍代码”，而是用一套固定的工程规范（P1–P11）对阶段二的产物逐项审计，并拿到可复核的证据。P1–P6 是规范类（规范本身是否健全、是否可自动校验），P7–P10 是测试类（分层测试是否到位），P11 是门禁类（CI 能否真的阻断不合规合并）。P1–P6 是规范类（规范本身是否健全、是否可自动校验），P7–P10 是测试类（分层测试是否到位），P11 是门禁类（CI 能否真的阻断不合规合并）。

编号	规范主题	产出物	一句话验收
P1	通用编码规范（命名/风格/结构）	《编码规范》Markdown	≥80% 规则可映射到 ESLint/Prettier 实际规则
P2	Git 提交与分支工作流规范	规范 + commitlint.config + PR 模板	提交格式可被 commitlint 直接校验
P3	Lint/Format 工具链与自动化落地	.eslintrc / .prettierrc / lint-staged 配置	本地与 CI 规则一致；lint 失败阻断 CI
P4	文档与注释规范	规范 + README 模板 + JSDoc 示例	API 文档可自动生成
P5	Code Review 规范	规范 + 可勾选 Checklist	有流程时序、评论分级、时延 SLA
P6	开源治理与发布规范	LICENSE / 贡献指南 / 模板 / 发布工作流	可实现半自动发布
P7	测试策略与测试金字塔	策略文档	分层有边界定义、工具选型有理由
P8	单元测试规范	规范 + 示例代码	有命名/结构/断言/mock 边界/反模式警示
P9	集成测试规范	规范 + 示例	有范围、替身策略、数据隔离、防 flaky 准则
P10	E2E 测试规范	规范 + 示例用例	有选取原则、稳定选择器、等待重试、报告机制
P11	CI/CD 流水线与质量门禁	流水线 YAML + 说明文档	阶段完整、有缓存并行、覆盖率门禁、可阻断合并

Hermes 的角色边界 Hermes 的职责是「编排 + 验证」：它负责调用相应工具把每项规范跑成可判定的结果（例如 P1 的 lint 规则覆盖率、P11 的门禁是否会真的 fail），并把结果固化成审计报告。Hermes 本身不是 lint 工具，也不替代具体技术栈的工具链。

P1 通用编码规范（命名/风格/结构）

规范定义（Role / Context / Task）

字段	内容
Role	资深技术规范制定者 / 工程效率工程师
Context	需要为项目制定一套团队级编码规范，作为 lint 配置、Code Review 依据、新人 onboarding 的单一事实来源。默认 TypeScript + Node.js 全栈。
Input	无（或已有规范草稿）

字段	内容
Output Format	Markdown，每条含「规则 / 反例 / 正例 / 对应 lint 规则」
Constraints	每条规范必须可被工具校验（落到 lint 规则）；不引入虚构工具；给出正例/反例。
Acceptance Criteria	命名/风格/结构三类齐全；每条有正反例；≥80% 规则可映射到 ESLint/Prettier 实际规则；可直接被团队采纳。

Task 要点

1. 命名规范：变量/函数/类/常量/文件/目录/数据库字段/API 路径
2. 代码风格：缩进、引号、分号、换行、行宽、import 顺序
3. 目录结构：分层与模块边界（如 src/domain、src/infra、src/ui）
4. 语言特性约束：TS 禁用 any、禁用 var、优先 const、显式返回类型
5. 错误处理与日志规范
6. 安全基线：输入校验、敏感信息不入库/不打印、依赖漏洞审计

Hermes 执行层：怎么跑、怎么验证

- 用 Hermes 把规范文档与 lint 配置做**规则映射核对**：统计规范条目数、其中能落到 ESLint/Prettier 规则名的条目数，算出可映射率并断言 ≥80%。
- 反向检查：扫描 lint 配置中的每条规则，确认规范文档里有对应说明（避免“配了规则但没写进规范”）。
- 抽查正反例：把文档里的反例代码片段写入临时文件跑一次 ESLint，必须报出至少一条错误（证明反例确实违规、正例确实通过）。

► P1 验证命令（示例，按项目实际技术栈替换）

```
# 1) 统计规范条目与可映射条目
python scripts\spec_audit.py --spec docs\CODING_STANDARD.md --lint .eslintrc.cjs

# 2) 反例必须报错、正例必须通过（真实执行 ESLint）
npx eslint --no-eslintrc -c .eslintrc.cjs tests\samples\anti-pattern.ts # 期望：有 error
npx eslint --no-eslintrc -c .eslintrc.cjs tests\samples\good-pattern.ts # 期望：无 error

# 3) 覆盖率断言
python -c "import
json;d=json.load(open('reports/p1_mapping.json'));print(d['mapped'],'/',d['total'], d['ratio'])"
```

P2 Git 提交与分支 workflow 规范

规范定义（Role / Context / Task）

字段	内容
Role	DevOps / 工程效率工程师
Context	规范提交信息与分支策略，支撑自动生成 CHANGELOG、可追踪、可回滚。
Input	无

字段	内容
Output Format	Markdown + <code>commitlint.config</code> 示例 + PR 模板
Constraints	默认 Conventional Commits; 明确说明两种分支策略适用场景; 给出 commitlint 配置示例。
Acceptance Criteria	提交格式有正反例; 分支策略有明确适用场景说明; 可被 commitlint 直接校验。

Task 要点

- 7. 提交信息规范 (Conventional Commits: type(scope): subject + body + footer)
- 8. 分支策略: trunk-based vs Git Flow 的取舍与适用团队规模
- 9. 分支命名: feature/xxx、fix/xxx、release/x.y.z、hotfix/xxx
- 10. PR/MR 流程: 标题规范、模板、关联 issue、rebase vs merge
- 11. 自动化强制: commitlint + husky (git hooks)

Hermes 执行层：怎么跑、怎么验证

- 用 commitlint 对 git 历史做回溯校验：对最近 N 条真实提交逐条跑 `commitlint`，输出通过率（这是硬证据，不是文档自述）。
- 构造正反例提交信息分别校验：正例必须通过，反例必须被拒（证明规则真的生效）。
- 核对 husky 钩子是否实际安装（`.husky/commit-msg` 存在且可执行），并用一次真实提交验证钩子被触发。

► P2 验证命令（示例，按项目实际技术栈替换）

```
# 1) 回溯校验最近 20 条真实提交
git log --format=%s -n 20 | ForEach-Object { echo $_ } | npx commitlint --from HEAD~20

# 2) 正反例
echo "feat(dedup): add two-level dedup engine" | npx commitlint # 期望: 通过
echo "update code" | npx commitlint # 期望: 失败

# 3) 钩子是否真的装上
Test-Path .husky/commit-msg
```

P3 Lint/Format 工具链与自动化落地

规范定义 (Role / Context / Task)

字段	内容
Role	工程效率 / 前端工程化工程师
Context	把 P1 规范自动化落地为 lint/format 工具链 + git hooks，保证本地与 CI 一致。
Input	P1 编码规范
Output Format	<code>.eslintrc</code> / <code>.prettierrc</code> / lint-staged 配置 + 集成说明 Markdown
Constraints	工具为真实存在（版本以官方文档为准，不写死未来版本）；格式化与质量职责分离（Prettier 管格式、ESLint 管质量）。
Acceptance Criteria	本地与 CI 规则一致；提交时自动修复格式；lint 失败能阻断 CI。

Task 要点

- 12. 工具选型：ESLint + Prettier + Stylelint（TS 生态）
- 13. 配置文件与规则集（推荐 preset，与 P1 逐条对应）
- 14. husky + lint-staged 配置（提交前自动格式化）
- 15. 编辑器集成：VS Code 推荐扩展 + settings.json
- 16. CI 中的 lint 检查与失败策略

Hermes 执行层：怎么跑、怎么验证

- **本地与 CI 一致性核验：**对比本地 lint 命令与 CI workflow 中的 lint 命令（含配置文件路径、node 版本），不一致即判不通过。
- **提交时自动格式化验证：**故意提交一个格式错误的文件，观察 lint-staged 是否自动修复并让提交成功（`git show --stat HEAD` 看格式化改动是否被带进提交）。
- **CI 阻断验证：**在分支上引入一处 lint error 推上去，确认 CI 在 lint 阶段 fail 且后续 e2e/deploy 阶段**未被执行**（这是“能阻断”的唯一硬证据）。

► P3 验证命令（示例，按项目实际技术栈替换）

```
# 1) 本地 vs CI 一致性
Select-String -Path .github\workflows\*.yaml -Pattern "lint" -Context 0,2
npm run lint           # 本地命令必须与 CI 中调用的命令等价

# 2) 自动格式化是否生效
"const x=1" | Out-File -Encoding utf8 scratch.ts ; git add scratch.ts
git commit -m "chore: scratch (should be auto-formatted)"
git show --stat HEAD    # scratch.ts 应以格式化后的形态进入提交

# 3) CI 阻断（真实跑一次）
git push origin ci/lint-fail-test # 观察 Actions: lint step 红, e2e/deploy step 未执行
gh run list --limit 3
```

P4 文档与注释规范

规范定义（Role / Context / Task）

字段	内容
Role	技术写作 / 文档工程师
Context	统一代码注释、API 文档、README 写法，覆盖开源与内部项目。
Input	无
Output Format	Markdown 规范 + README 模板 + JSDoc 示例
Constraints	明确注释语言约定（默认英文注释 + 中文可选的说明）；README 模板可直接复用。
Acceptance Criteria	有 README 标准结构；注释有正反例；API 文档可自动生成。

Task 要点

17. 注释规范：何时写注释、TODO/FIXME 约定、禁止废话注释
18. 内联文档：JSDoc/TSDoc (TS) 或 docstring (其他语言)
19. README 标准结构模板：简介/安装/快速开始/API/贡献/许可证
20. API 文档生成：TypeDoc / OpenAPI(Swagger)
21. 开源附加：CONTRIBUTING.md、CODE_OF_CONDUCT、LICENSE 说明

Hermes 执行层：怎么跑、怎么验证

- **API 文档可自动生成**：真实跑一次 TypeDoc/OpenAPI 生成器，断言产物目录非空且文件数 > 0（生成失败是最常见的“文档规范只写在纸上”）。
- **README 结构核验**：按模板章节清单逐项检查目标 README 是否存在对应标题，输出缺失章节列表。
- **注释反例扫描**：用 lint（如 eslint-plugin-jsdoc）扫描代码库，统计缺失 JSDoc 的公共导出数量。

► P4 验证命令（示例，按项目实际技术栈替换）

```
# 1) 文档可生成
npx typedoc --out docs/api src/index.ts ; (Get-ChildItem docs/api -Recurse -File).Count # > 0

# 2) README 章节齐备性
python scripts\readme_section_check.py README.md

# 3) 公共导出缺注释统计
npx eslint src --format json > reports\p4_jsdoc.json
```

P5 Code Review 规范

规范定义 (Role / Context / Task)

字段	内容
Role	资深工程师 / Tech Lead
Context	需要一套可执行的 CR 流程与审查清单，提升质量、加速合并。
Input	P1、P2、P3 规范
Output Format	Markdown 规范 + 可复制的审查 Checklist
Constraints	区分小改动快速通道（如 <100 行可简化）；清单可勾选。
Acceptance Criteria	有流程时序、分级评论约定、可勾选清单、时延 SLA。

Task 要点

22. CR 流程：发起→指派→审查→修改→合并，角色与时延 SLA
23. 审查清单：正确性/安全/性能/可读性/测试/规范符合度
24. 评论分级：blocker（必须改）/ 建议 / nit（可忽略），友好沟通
25. 合并策略：需几个 approve、CI 绿灯、squash/rebase

26. 度量：CR 响应时长、revert 率

Hermes 执行层：怎么跑、怎么验证

- 用 Hermes 对现有 PR 做一次回溯评审：拉取最近 N 个已合并 PR，检查是否满足规范（有描述、关联 issue、CI 绿灯、reviewer 数达标、commit message 合规），输出符合率。
- 检查分支保护规则是否配置（gh api 读 branch protection），这是 CR 规范能否被强制的技术前提。
- 度量真实数字：CR 响应时长可从 PR 的 created→review 时间戳计算；revert 率可从 git 历史中 revert 提交占比计算。

► P5 验证命令（示例，按项目实际技术栈替换）

```
# 1) 回溯最近 10 个已合并 PR 的规范符合率
gh pr list --state merged --limit 10 --json number,title,body,reviews,mergeCommit

# 2) 分支保护是否真的开启（CR 能否被强制）
gh api repos/{owner}/{repo}/branches/main/protection --jq '.required_pull_request_reviews'

# 3) revert 率（实测）
git log --oneline -n 200 | Select-String -Pattern "^\\w+ revert" | Measure-Object
```

P6 开源治理与发布规范

规范定义（Role / Context / Task）

字段	内容
Role	开源项目维护者 / DevRel 工程师
Context	开源项目需要治理规范：LICENSE、贡献指南、Issue/PR 模板、版本与发布。
Input	P2 提交规范（配合自动发布）
Output Format	Markdown + 模板文件 + GitHub Actions 发布工作流示例
Constraints	许可证只给建议不给法律意见；发布工具为真实存在。
Acceptance Criteria	有 LICENSE/贡献指南/模板/CHANGELOG 规范；可实现半自动发布。

Task 要点

27. LICENSE 选择：MIT / Apache-2.0 / GPL 的取舍（仅建议，非法律意见）
28. CONTRIBUTING.md 与行为准则 CODE_OF_CONDUCT
29. Issue/PR 模板：bug report / feature request
30. 版本规范 SemVer + CHANGELOG（Keep a Changelog 格式）
31. 自动发布：semantic-release / Changesets + 徽章

Hermes 执行层：怎么跑、怎么验证

- 用 `gh api` 直接核对仓库治理文件是否**真实存在且内容正确**：LICENSE（与 package.json 中 license 字段是否一致）、CONTRIBUTING.md、`.github/ISSUE_TEMPLATE/`、`.github/PULL_REQUEST_TEMPLATE.md`。
- CHANGELOG 与 Release 一致性核对：每个 tag 是否在 CHANGELOG 中有对应条目，Release 的 tag 是否指向 main 的最新提交（这是最常见的翻车点）。
- 发布工作流存在性 + 最近一次运行结果（`gh run list --workflow=release.yml`）。

► P6 验证命令（示例，按项目实际技术栈替换）

```
# 1) 治理文件在位（逐项 Test-Path）
foreach ($f in @( 'LICENSE', 'CONTRIBUTING.md', 'CODE_OF_CONDUCT.md',
                  '.github/PULL_REQUEST_TEMPLATE.md' )) { "$f => " + (Test-Path $f) }

# 2) CHANGELOG 与 tag 对齐
git tag --list ; Select-String -Path CHANGELOG.md -Pattern "^## \[v"

# 3) Release 与最新提交是否同一提交（防 tag 滞后）
git ls-remote origin refs/tags/v1.0.0 refs/heads/main

# 4) 许可证一致性
gh api repos/{owner}/{repo} --jq '.license.spdx_id' ; Select-String -Path package.json -Pattern '"license"'
```

P7 测试策略与测试金字塔

规范定义（Role / Context / Task）

字段	内容
Role	测试架构师 / QA Lead
Context	为项目设计整体测试策略：分层、范围、工具、成本控制。
Input	无
Output Format	Markdown 测试策略文档
Constraints	比例是建议值可调整；工具为真实存在；强调“测行为不测实现”。
Acceptance Criteria	分层清晰、各层有边界定义、工具选型有理由、命名组织规范可落地。

Task 要点

32. 测试金字塔定义：单元/集成/E2E 比例建议（如 70/20/10）
33. 各层职责边界：测什么 / 不测什么
34. 工具选型：Jest/Vitest（单测）、pytest（如 Python）、Playwright/Cypress（E2E）
35. 测试环境与数据策略：fixture、mock 边界
36. 测试命名与目录组织规范

Hermes 执行层：怎么跑、怎么验证

- **实测分层比例**：分别统计单测/集成/E2E 的用例数量与执行耗时，算出真实比例与金字塔偏差（很多项目实际是“倒金字塔”或“沙漏形”）。

- 边界越界检查：扫描单测文件是否 import 了数据库/网络客户端（单测里出现真实 IO 即违反“测行为不测实现”的边界约定）。
- 工具选型核验：确认策略文档中选定的工具确实装在 devDependencies / requirements 中，且能真实跑起来（不做纸面选型）。

► P7 验证命令（示例，按项目实际技术栈替换）

```
# 1) 三层用例数量与耗时
npx vitest list --json > reports\p7_unit.json
python -m pytest --collect-only -q | Select-Object -Last 3

# 2) 单测越界检查（单测中不应出现真实 DB/网络）
Select-String -Path tests\unit\**\*.ts -Pattern "prisma|axios|fetch\(|pg\."

# 3) 工具是否真实安装
Get-Content package.json | Select-String -Pattern "vitest|playwright"
```

P8 单元测试规范

规范定义（Role / Context / Task）

字段	内容
Role	高级测试 / 开发工程师
Context	统一单元测试写法，保证可读、可维护、快速、确定。
Input	P7 策略
Output Format	Markdown + 示例代码
Constraints	测试必须确定性（无时间/随机依赖）；示例基于 Jest/Vitest。
Acceptance Criteria	有命名/结构/断言规范；有 mock 边界；有边界值与异常用例要求；有反模式警示。

Task 要点

- 命名约定：describe/it、Given-When-Then、测试描述语义
- AAA（Arrange-Act-Assert）结构
- 断言与匹配器使用规范
- Mock/Stub 边界：何时 mock、避免过度 mock
- 边界值 / 异常路径用例要求
- 快照测试的使用与滥用约束

Hermes 执行层：怎么跑、怎么验证

- **确定性核验（硬指标）**：把同一套单测连续跑 N 次（如 5 次），全绿且结果一致才算通过；出现任何一次 flaky 即判不通过并记录用例名。
- 时间/随机依赖扫描：grep `Date.now()`、`new Date()`、`Math.random()`、`setTimeout()` 在单测中的出现，未使用假时钟即列为风险项。

- 反模式扫描：统计 `toBeTruthy()` 这类弱断言占比（弱断言过多说明断言不规范）。
- 覆盖边界值：检查测试文件是否包含空/边界/异常路径的显式用例名。

► P8 验证命令（示例，按项目实际技术栈替换）

```
# 1) 连跑 5 次验确定性
1..5 | ForEach-Object { npx vitest run --reporter=dot 2>&1 | Select-String "Tests" }

# 2) 时间/随机依赖
Select-String -Path tests\unit\**\*.ts -Pattern "Date\.now\(|Math\.random\(|setTimeout\("

# 3) 弱断言占比
python scripts\weak_assert_scan.py tests\unit
```

P9 集成测试规范

规范定义（Role / Context / Task）

字段	内容
Role	测试工程师
Context	规范跨模块、跨服务、与真实依赖（DB、消息队列、外部 API）的集成测试。
Input	P7 策略
Output Format	Markdown + 示例
Constraints	推荐 testcontainers 等真实工具；强调防 flaky。
Acceptance Criteria	有范围界定、替身策略、数据隔离方案、防 flaky 准则。

Task 要点

43. 集成测试范围：模块间接口、DB、缓存、外部服务
44. 测试替身策略：testcontainers / fake / stub
45. 数据隔离与清理：每测试独立、事务回滚、schema 迁移
46. 异步与并发测试注意事项
47. 稳定性：防 flaky（重试、超时、等待策略）

Hermes 执行层：怎么跑、怎么验证

- **数据隔离核验**：并行执行集成测试（如 `--shard` 或多进程），若出现互相干扰即说明隔离不到位；同时检查测试后数据库是否有残留数据。
- 不依赖“手动准备好环境”：在一台干净环境里从零跑集成测试，断言能被自动拉起（testcontainers 起容器成功）而不是连不上库。
- 防 flaky 措施核验：检查是否存在无上限重试、固定 sleep 等反模式；连续跑 3 次验证稳定性。

► P9 验证命令（示例，按项目实际技术栈替换）

```
# 1) 并隔离性（并行跑两次，结果应一致）
python -m pytest tests/integration -q -n 4
python -m pytest tests/integration -q -n 4

# 2) 残留数据检查
python -m pytest tests/integration -q ; python scripts\db_row_count.py # 期望回到基线

# 3) 反模式扫描
Select-String -Path tests\integration\**\*.py -Pattern "time\.sleep\(|retries="
```

P10 E2E 测试规范

规范定义（Role / Context / Task）

字段	内容
Role	QA / 自动化测试工程师
Context	规范端到端测试（用户视角），覆盖关键用户路径。
Input	P7 策略
Output Format	Markdown + 示例用例
Constraints	强调可维护性与防 flaky；选择器用稳定策略。
Acceptance Criteria	有用例选取原则、稳定选择器规范、等待/重试策略、报告机制。

Task 要点

48. 工具选型：Playwright / Cypress
49. 用例选取原则：关键路径 smoke、核心业务 flow，避免穷举
50. 选择器规范：data-testid 优先，避免脆弱 XPath
51. 等待与重试策略：auto-wait、避免 sleep
52. 环境与数据准备、并行执行、报告与截图

Hermes 执行层：怎么跑、怎么验证

- **选择器稳定性扫描**：统计 E2E 用例中 XPath / CSS 结构选择器（`div > div:nth-child`）与 `data-testid` 的比例，结构选择器占比过高即判可维护性风险。
- flaky 实测：把 E2E 套件连续跑 3 次（Playwright 可用 `--repeat-each=3`），统计不稳定用例名与失败率。
- 产物核验：跑完后确认报告与截图/trace 文件真实生成（失败时没有 trace/chunk 会让排查成本极高）。
- 大陆网络注意：Playwright 浏览器官方 CDN 卡顿，用
`PLAYWRIGHT_DOWNLOAD_HOST=https://cdn.npmmirror.com/binaries/playwright` 或 `channel: 'msedge'` 复用系统 Edge 免下载。

► P10 验证命令（示例，按项目实际技术栈替换）

```
# 1) 选择器稳定性
python scripts\selector_audit.py e2e\ # 输出 data-testid 占比与脆弱选择器清单
```

```
# 2) 连续 3 次验 flaky
npx playwright test --repeat-each=3 --reporter=line

# 3) 报告与 trace 真实产出
(Get-ChildItem playwright-report -Recurse -File).Count
(Get-ChildItem test-results -Recurse -File -Filter *.zip).Count    # trace 包
```

P11 CI/CD 流水线与质量门禁

规范定义（Role / Context / Task）

字段	内容
Role	DevOps / CI 工程师
Context	把 lint、测试、覆盖率、构建、发布串成 CI 流水线，设置质量门禁。
Input	P3、P6、P8–P10
Output Format	流水线 YAML 示例 + 说明文档
Constraints	以 GitHub Actions 为例（注明可迁移 GitLab CI/Jenkins）；门禁阈值给建议值（全局 80%、增量 ≥90%）。
Acceptance Criteria	阶段完整、有缓存/并行、有覆盖率门禁、有失败通知、能阻断不合规合并。

Task 要点

53. 流水线阶段：checkout→install→lint→unit→integration→e2e→build→deploy
54. 缓存与并行加速策略
55. 覆盖率门禁：阈值、增量覆盖率、Codecov 报告
56. 失败处理与通知
57. 发布流水线（配合 P6 自动发布）

Hermes 执行层：怎么跑、怎么验证

- **门禁有效性验证（最关键）**：故意引入不合规改动（lint error / 失败测试 / 覆盖率下降），推分支跑 CI，断言流水线必须失败、且后续阶段未执行。门禁“只写在 YAML 里但不会真的 fail”是最常见的假门禁。
- 阶段完整性核验：读 workflow YAML，逐项检查约定的 8 个阶段是否存在、顺序是否正确、是否有 **needs** 依赖。
- 缓存与并行：检查是否配置 **actions/cache/setup-node** 的 cache，以及矩阵或拆分 job 的并行；用最近一次 run 的真实时长做对比证据。
- 覆盖率门禁：检查阈值配置（全局 80%、增量 ≥90%）并确认 CI 中确实会读取该配置（阈值配了但没接上同样无效）。
- 通知：检查失败通知渠道（Actions 默认邮件 / Slack webhook / **if: failure()** 步骤）是否真实存在且可触达。

► P11 验证命令（示例，按项目实际技术栈替换）

```
# 1) 阶段完整性与顺序
Select-String -Path .github\workflows\ci.yml -Pattern "^\s{2}\w+:|needs:|uses: actions/cache"

# 2) 门禁真的会 fail (推一个注定失败的改动)
git checkout -b ci/gate-test ; "const x:any=1" | Out-File -Append src\index.ts
git commit -am "test: gate should fail" ; git push origin ci/gate-test
gh run watch      # 期望: lint job 失败; unit/e2e 未执行

# 3) 最近运行的阶段耗时 (缓存效果证据)
gh run list --limit 5 --json databaseId,conclusion,createdAt,updatedAt

# 4) 覆盖率门禁配置是否接上
Select-String -Path .github\workflows\*.yml -Pattern "coverage|codecov|80|90"
```

4.12 阶段三的产出：验证报告应包含什么

章节	必含内容	证据形式
P1-P6 规范审计	每项规范的健全性评级 + 可映射率 + 缺口清单	命令输出截图/文本、统计数字
P7-P10 测试审计	三层用例数量与耗时、真实分层比例、越界项、flaky 复现记录	连续多次运行的通过率数字
P11 门禁审计	阶段完整性、门禁是否真的能 fail、缓存命中与耗时对比	CI 运行链接 + 失败 run 的阶段截图
合规性结论	逐项 PASS / PARTIAL / FAIL 评级与理由	评级表 (必须带判定依据)
改进项	按优先级排序的整改清单 (含负责人与验收方式)	可勾选任务清单
诚实声明	未验证项、样本不足项、受限项 (如无沙箱环境)	明确写出, 不得用推断充数

铁律 验证报告的铁律：**每一项结论都要有可复现的命令或可核对的文件作为依据**。无证据时如实写“未验证”，绝不编造数字充数。

第五章 风险提示

5.1 六大风险与对策

#	风险	严重度	具体表现	对策
R1	规范过重风险	最高	规范条目过多 → 官僚化 → 开发者 绕过程序 → 效率反降	优先纳入“可自动校验”的规则（lint/测试/CI 能判定的）； 人工规则精简为少数核心项；先落地再扩 充
R2	工具版本时效	高	把版本号写死 → 配置在半年后全面 过期， 照抄即报错	不写死版本号，引用官方文档“以最新稳定 版为准”； 配置只保留结构性内容；本文档即按此原 则编写
R3	测试反模式	高	100% 覆盖率目标陷阱、快照滥 用、 flaky 测试腐蚀测试信心	覆盖率看增量而非绝对；快照仅用于稳定 结构； P8/P9/P10 中对三类反模式明确警示并给出 检测手段
R4	多语言项目不一致	中	同一团队多语言时强行套一套规 范， Java 项目抄 TS 规则	规范分层：通用层（命名/提交/CR/测试原 则） + 语言适配层（工具链、语法约束）
R5	开源与内部侧重点 不同	中	把内部效率优先的规范套到开源项 目， 贡献门槛过高、无人敢提 PR	内部重效率与 CR；开源重贡献门槛、许可 证、 发布可追溯。两套模板分开维护，勿混用
R6	门禁过严阻塞交付	中	覆盖率阈值一开始就设 90%， 导致所有 PR 卡住、团队开始绕过 CI	初期以“增量覆盖率”为主（新增代码 ≥90%）、 全局为辅（如 80%），渐进提升； 给核心链路留豁免清单并标注原因

5.2 AI 编码本身的风险（容易被忽略）

风险	表现	对策
臆造 API / 库版本	编出不存在的函数名、参数、配置项， 且语气非常笃定	要求“不臆造，不确定就标【需确认】”； 执行后必须真实跑一次（编译/测试）而非通读代 码
静默失败	抓取返回 0 条不报错、 迁移改了字段但没改调用方	为每个环节设“非空/非零断言”； 关键路径加心跳与告警
权限过大	<code>--dangerously-skip-permissions</code> 用于 日常开发， AI 误删文件或外传内容	默认模式逐步确认；沙箱环境才用跳过权限； 敏感文件加入忽略清单
密钥泄漏	AI 把 token 写进代码、日志或提交信息	密钥只走环境变量；提交前用 gitleaks 之类工具 扫描； CI 中加 secret scanning
上下文漂移	长会话中 AI 逐渐偏离最初的约束	每完成一个子任务就开新会话喂新 Prompt； 把约束写进 <code>CLAUDE.md</code> 等项目级常驻文件

风险	表现	对策
无审查合并	AI 说完成就直接合并，无人复核 diff	合并前必看 <code>git diff --stat</code> + 关键文件 diff; CR 规范 (P5) 强制至少 1 个 approve

5.3 上线前检查清单

- ☐ 每条 Prompt 的七要素是否齐备 (Role/Context/Task/Constraints/Input/Format/Acceptance) ?
- ☐ 每个子任务的验收标准是否可以用**一条命令或一个数字**判定?
- ☐ AI 是否真实跑过测试/编译 (而不是只在文字里声称通过) ?
- ☐ `git diff --stat` 的改动范围是否与任务边界一致 (有没有动不该动的文件) ?
- ☐ 是否有新增依赖? 新增依赖是否已在 CHANGELOG/文档中说明?
- ☐ 是否有任何密钥、token、内网地址被写入代码或提交信息?
- ☐ P1-P11 中哪些项已 PASS、哪些是 PARTIAL、哪些明确未验证? 未验证项是否已在报告中如实标注?
- ☐ CI 门禁是否验证过“真的会失败” (而不是只存在于 YAML 里) ?
- ☐ 回滚方案是否存在 (能否一条命令 `revert` 到上一个可用版本) ?
- ☐ README / CHANGELOG 是否已同步更新, 新人能否照着文档独立跑通?

附录

附录 A Prompt 七要素速查卡

► 复制到每条 Prompt 里，逐项填满

【Role】	你是谁：一句话说清执行者的专业身份
【Context】	为什么做：项目现状 + 技术栈 + 本任务在链路中的位置
【Task】	做什么：只写一件事，动词开头，可数可判定
【Constraints】	不做什么：依赖限制 / 不可改动的文件 / 兼容性 / 语言与风格约定
【Input】	从哪来：确切文件路径 + 数据结构字段名（要求先读文件再动手）
【Format】	交什么：完整文件 / diff / JSON / 表格，以及附带的简短说明
【Acceptance】	怎样算完：可用命令或数字判定的条件，写清必测用例清单

附录 B Hermes 阶段三验证命令清单（速查）

检查目的	命令	期望结果
环境与认证	<code>claude --version / claude auth status</code>	输出版本号；loggedIn: true
改动范围审计	<code>git diff --stat / git status --short</code>	改动文件与任务边界一致；无意外文件
提交规范	<code>npx commitlint --from HEAD~20</code>	通过率 100%（或列明例外）
单测确定性	单测连跑 5 次	5/5 全绿且结果一致
E2E flaky	<code>npx playwright test --repeat-each=3</code>	无不稳定用例
PDF/文档产物	<code>pymupdf</code> 打开并 <code>page.get_text()</code> 搜关键词	中文可被抽出（证明非方块/非图片化）
CI 门禁有效性	推一个注定失败的改动 → <code>gh run watch</code>	流水线失败，后续阶段未执行
Release 对齐	<code>git ls-remote origin refs/tags/vX</code> <code>refs/heads/main</code>	tag 与 main 指向同一提交
远程与本地同步	<code>gh api repos/o/r/git/trees/main</code> 与本 <code>git ls-files</code> 对比	条目数一致

附录 C 目录结构与术语表

推荐的项目目录结构（把任务档案与验收产物一并纳管）：推荐的项目目录结构（把任务档案与验收产物一并纳管）：

► 推荐目录结构

project/	
├─ tasks/	# 阶段① 产出：任务卡与 Prompt 档案
│ └─ T-01-data-model.md	
│ └─ T-03-dedup.md	
│ └─ README.md	# 任务总表与依赖 DAG
├─ docs/	
│ └─ CODING_STANDARD.md	# P1
│ └─ GIT_WORKFLOW.md	# P2
│ └─ DOC_STANDARD.md	# P4
│ └─ CODE_REVIEW.md	# P5
│ └─ TEST_STRATEGY.md	# P7

```
|   └─ AI-Vibe-Coding-使用指南.docx    # 本文档
├─ reports/                            # 阶段③ 产出：验证证据与审计报告
|   └─ p1_mapping.json
|       └─ AUDIT_REPORT.md
├─ src/  tests/  e2e/
├─ .github/workflows/                 # P11
├─ CHANGELOG.md                       # P2 / P6
└─ README.md
```

术语	含义
Vibe-Coding	以对话驱动 AI 生成代码的开发方式；本指南把它工程化、流程化
任务卡 / Prompt 套件	阶段① 产出的结构化任务描述，每条含七要素，可直接投喂执行层 AI
Code Architect 角色	Chatbox 中的「代码架构师」人格设定，负责复述、追问、拆解、排序、生成 Prompt
验收标准 (Acceptance Criteria)	可用命令行或数字判定的完成条件；不可判定的标准视为不合格
质量门禁 (Quality Gate)	CI 中能真实阻断不合规合并的检查点；只写在 YAML 里而不生效的称为“假门禁”
测试金字塔	单测：集成：E2E 的建议比例（约 70/20/10），越底层越多、越快、越便宜
flaky 测试	同一代码反复运行结果不稳定的测试；会腐蚀团队对测试的信任，必须定位或隔离
PARTIAL / FAIL 评级	验证报告中用于标记“部分满足”与“不满足”的结论等级，必须附判定依据

— 全文完 —

MIT License · Copyright (c) 2026 Misaka4396